

CS336 Assignment 2 Systems 实现总结

李东泽

2026 年 6 月 5 日

1 完成内容概览

本次完成了 Assignment 2 中公开测试覆盖的全部系统组件，并将测试适配器连接到 `cs336_systems` 模块。主要新增或修改文件如下：

文件	作用
<code>cs336_systems/flash_attention.py</code>	实现 PyTorch autograd 版本 FlashAttention，以及调用 Triton kernel 的 CUDA forward/backward 版本。
<code>cs336_systems/distributed.py</code>	实现 individual-parameter DDP 和 bucketed DDP，包括初始化广播、反向 hook、梯度同步和 bucket scatter。
<code>cs336_systems/sharded_optimizer.py</code>	实现 optimizer state sharding：每个 rank 只维护一部分参数的 optimizer state，更新后广播参数。
<code>tests/adapters.py</code>	将官方测试入口改为薄适配层，返回或调用 <code>cs336_systems</code> 中的实现。
<code>cs336_systems/__init__.py</code>	增加源码目录直接运行时的版本号 fallback，避免未安装包时导入失败。

2 FlashAttention

2.1 原理

标准 attention 先显式构造分数矩阵

$$S = \frac{QK^\top}{\sqrt{d}}, \quad P = \text{softmax}(S), \quad O = PV.$$

显式保存完整的 S 和 P 会占用 $O(BN^2)$ 显存。FlashAttention 的核心思想是用分块方式计算 softmax，并只保存反向传播所需的 log-sum-exp：

$$L_i = \log \sum_j \exp(S_{ij}).$$

在反向传播时可以通过

$$P_{ij} = \exp(S_{ij} - L_i)$$

重构 softmax 概率，不需要保存完整概率矩阵。反向传播中使用：

$$dV = P^\top dO, \quad dS = P \odot \left(dP - \sum_j P_{ij} dP_{ij} \right), \quad dQ = dSK/\sqrt{d}, \quad dK = dS^\top Q/\sqrt{d}.$$

本实现包含两个入口：

- **FlashAttentionPytorch**: 只使用 PyTorch tensor op，实现清晰、便于验证。
- **FlashAttentionTriton**: 在 CUDA tensor 上调用自定义 Triton forward/backward kernel；无 Triton 或 CPU 环境下 fallback 到 PyTorch 版本。

2.2 关键代码

PyTorch forward 保存 q,k,v,lse，其中测试要求恰好保存一个形状为 (batch, n_queries) 的 log-sum-exp tensor：

```
scores = _attention_scores(q, k, bool(is_causal))
lse = torch.logsumexp(scores, dim=-1)
p = torch.exp(scores - lse.unsqueeze(-1))
out = torch.matmul(p, v)

ctx.save_for_backward(q, k, v, lse)
```

Triton forward kernel 以每个 batch/query 为一个 program，计算一行 attention：

```
scores = tl.sum(k * q[None, :], axis=1) * scale
scores = tl.where(valid, scores, -1.0e6)

row_max = tl.max(scores, axis=0)
exp_scores = tl.exp(scores - row_max)
exp_sum = tl.sum(exp_scores, axis=0)
lse = tl.log(exp_sum) + row_max
p = exp_scores / exp_sum

out = tl.sum(p[:, None] * v, axis=0)
```

Triton backward kernel 中，dq 属于当前 query，可以直接写回；dk/dv 被多个 query 共同贡献，因此使用 atomic add：

```
dp = tl.sum(v * do[None, :], axis=1)
delta = tl.sum(p * dp, axis=0)
ds = p * (dp - delta)

dq = tl.sum(ds[:, None] * k, axis=0) * scale
dk = ds[:, None] * q[None, :] * scale
dv = p[:, None] * do[None, :]
```

```
tl.atomic_add(grad_k_ptr + ..., dk, sem="relaxed", mask=...)
tl.atomic_add(grad_v_ptr + ..., dv, sem="relaxed", mask=...)
```

3 Individual-Parameter DDP

3.1 原理

Distributed Data Parallel 的目标是让每个 rank 处理不同 mini-batch shard，但参数更新等价于单机大 batch。具体做法是：

1. 初始化时从 rank 0 广播完整模型状态，确保所有 rank 参数一致。
2. 每个可训练参数注册 backward hook。
3. 某个参数的梯度就绪后立即执行 `all_reduce(sum)`。
4. optimizer step 前等待所有通信完成，并将梯度除以 `world_size`。

这样每个 rank 的梯度变为全局平均梯度，因此每个 rank 的 optimizer step 与单机全量 batch 一致。

3.2 关键代码

```
for tensor in self.module.state_dict().values():
    _broadcast_(tensor, src=0)

for param in _unique_trainable_parameters(self.module):
    param.register_post_accumulate_grad_hook(self._make_hook(param))
```

```
def hook(_param):
    if param.grad is None or self._world_size == 1:
        return
    work = _all_reduce_sum_(param.grad, async_op=True)
    self._handles.append((work, param.grad))

def finish_gradient_synchronization(self):
    for work, grad in self._handles:
        work.wait()
        grad.div_(self._world_size)
    self._handles.clear()
```

实现中还处理了 `gloo + CUDA tensor` 的情况：官方测试使用 `gloo backend`，但 `CUDA` 可见时模型会被放到 `GPU`。对于 `gloo` 不直接支持的 `CUDA tensor`，代码先复制到 `CPU` 做 `collective`，再复制回原设备。

4 Bucketed DDP

4.1 原理

Individual-parameter DDP 每个参数一次通信，通信次数多。Bucketed DDP 将多个参数梯度拼接成 bucket，在 bucket 内所有参数梯度都就绪后发起一次 all-reduce，从而减少通信启动开销。

本实现步骤如下：

1. 根据 `bucket_size_mb` 按参数顺序建立 bucket。
2. 每个参数的 hook 标记所在 bucket 中该参数已经 ready。
3. 当 bucket 中所有参数 ready 后，将梯度 flatten 后拼接为一个 tensor，并异步 all-reduce。
4. optimizer step 前等待通信完成，将平均后的 flat gradient scatter 回各参数的 `.grad`。

4.2 关键代码

构建 bucket：

```
for param in params:
    param_size = param.numel() * param.element_size()
    if current and max_size is not None and current_size + param_size > max_size:
        self._append_bucket(current)
        current = []
        current_size = 0
    current.append(param)
    current_size += param_size
```

bucket ready 后发起同步：

```
bucket.ready.add(id(param))
if len(bucket.ready) == len(bucket.params) and bucket.work is None:
    self._launch_bucket(bucket)

grads = [param.grad.reshape(-1) for param in bucket.params]
bucket.flat_grad = torch.cat(grads)
bucket.work = _all_reduce_sum_(bucket.flat_grad, async_op=True)
```

同步完成后 scatter 回原参数：

```
bucket.work.wait()
bucket.flat_grad.div_(self._world_size)
offset = 0
for param in bucket.params:
    numel = param.numel()
    param.grad.copy_(bucket.flat_grad[offset : offset + numel].view_as(param))
    offset += numel
```

5 Sharded Optimizer

5.1 原理

普通 AdamW 在每个 rank 上都保存所有参数的 optimizer state，例如一阶动量和二阶动量。Optimizer state sharding 的目标是让每个 rank 只保存一部分参数的 optimizer state，降低每张卡的内存压力。

本实现采用简单稳定的参数分片策略：

$$\text{owner}(i) = i \bmod \text{world_size}.$$

每个 rank 只把自己负责的参数传给本地 optimizer。执行 `step()` 后，每个参数由其 owner rank 广播到所有 rank，从而恢复所有模型副本的一致性。

5.2 关键代码

```
def _owner(self, param_index: int) -> int:
    return param_index % self.world_size

local_group["params"] = [
    param for param in group["params"]
    if self._owner(self.param_to_index[id(param)]) == self.rank
]
self.local_optimizer = optimizer_cls(local_groups, **kwargs)
```

```
if self.local_optimizer is not None:
    self.local_optimizer.step()

for index, param in enumerate(self.global_params):
    _broadcast_(param.data, src=self._owner(index))
```

6 测试数据与结果

6.1 测试环境

最终 GPU 测试没有使用新建的 uv 环境，因为服务器根分区空间不足，安装 CUDA 版 torch 时失败。改用服务器已有的 rk-gemm conda 环境，其 PyTorch 版本与作业依赖匹配：

- Python 环境：/home/ldz/miniconda3/envs/rk-gemm/bin/python
- PyTorch：2.6.0+cu124
- GPU 限制：CUDA_VISIBLE_DEVICES=0,2,4,5
- 可见 GPU：4 张 NVIDIA GeForce RTX 3090

6.2 公开测试结果

测试	覆盖内容	结果
CUDA full tests	FlashAttention PyTorch/Triton、bucketed DDP、individual DDP、sharded optimizer，全 CUDA 可见路径。	16 passed
CPU-only tests	CPU-only 路径；CUDA/Triton attention 测试按 pytest 条件跳过。	12 passed, 4 skipped
compileall	Python 语法和 import 基本检查。	passed

完整测试命令如下：

```
CUDA_VISIBLE_DEVICES=0,2,4,5 \
/home/ldz/miniconda3/envs/rk-gemm/bin/python -m pytest tests -q

CUDA_VISIBLE_DEVICES= python -m pytest tests -q

python -m compileall cs336_systems tests/adapters.py
```

CUDA 全套测试输出摘要：

```
tests/test_attention.py::test_flash_forward_pass_pytorch PASSED
tests/test_attention.py::test_flash_forward_pass_triton[False] PASSED
tests/test_attention.py::test_flash_forward_pass_triton[True] PASSED
tests/test_attention.py::test_flash_backward_pytorch PASSED
tests/test_attention.py::test_flash_backward_triton[False] PASSED
tests/test_attention.py::test_flash_backward_triton[True] PASSED
tests/test_ddp.py::* PASSED
tests/test_ddp_individual_parameters.py::* PASSED
tests/test_sharded_optimizer.py::* PASSED

16 passed, 1 warning in 47.17s
```

6.3 Benchmark 数据记录

仓库中已有当前硬件的 RTX 4090 benchmark 结果，位于 `benchmark_outputs/2026-04-27_rtx4090_formal/RESULTS.md`。其中 FP32 baseline timing 如下，可作为系统优化报告中的性能参考数据：

模型	fwd 均值/ms	fwd 标准差/ms	fwd+bwd 均值/ms	fwd+bwd 标准差/ms
small	11.448	0.066	35.383	0.514
medium	23.370	0.088	68.459	1.153
large	39.026	0.123	120.142	8.285

xl	73.647	0.129	200.166	1.484
2.7b	107.876	1.129	OOM	—

这组数据说明：在 24GB RTX 4090 上，2.7B 模型的 forward 可以运行，但 forward+backward 在 batch size 4、context length 128、vocab size 10000 的设定下会 OOM。

7 结论

Assignment 2 的核心系统组件已经完成并通过公开测试。实现上遵循了作业要求：

- FlashAttention 保存 LSE 并在 backward 中重构 softmax；CUDA 路径实际调用 Triton kernel。
- DDP 初始化广播模型参数，并在 backward 阶段同步梯度，保证与单机大 batch 训练一致。
- Bucketed DDP 将多个参数梯度合并通信，降低 per-parameter all-reduce 的启动开销。
- Sharded optimizer 将 optimizer state 分散到不同 rank，step 后广播参数维持副本一致性。

最终验证结果为 CUDA 可见环境下 16 passed, CPU-only 环境下 12 passed, 4 skipped。